

Modül 15 — Rehberli Laboratuvar

Quarto ile Raporlama

Prof. Dr. Hakan Mehmetcik

2026-09-13

Bu Belge Nedir?

Bu bir **sınav provasası değildir**; ders saatinde birlikte, bilgisayar başında yapılacak bir uygulama defteridir. Notlandırılmaz.

Bu lab diğerlerinden bir noktada ayrılır: burada yalnızca kod yazmıyor, **bir belge kuruyorsunuz**. Bu yüzden görevlerin bir kısmı, bu dosyanın içinde değil, kendi oluşturacağınız `ilk-raporum.qmd` dosyasında yapılacaktır.

Belge iki bölümden oluşur:

- **Görevler** (Bölüm A–F): her görevde ne yapmanız gerektiği yazılıdır. Kod iskeletleri **eksik bırakılmıştır** ve render sırasında çalıştırılmaz.
- **Çözümler** (belgenin sonu): tüm görevlerin çalışan hâli. Önce kendiniz deneyin, tıkanmışınızda bakın.

💡 Nasıl çalışılır

Bu labdaki en önemli düğme **Render**'dır. Her görevden sonra belgenizi render edin. Hata alırsanız iyi haber: hatayı **iki satır kod sonra** yakaladınız, elli satır sonra değil.

Bölüm A — Belgeyi kurmak

Görev A1 — Yeni bir Quarto belgesi

RStudio'da **File > New File > Quarto Document** yolunu izleyin. Başlık olarak “İlk Raporum” yazın, biçim olarak **PDF** seçin. Dosyayı `ilk-raporum.qmd` adıyla, bu modülün klasörüne kaydedin.

Açılan belgedeki örnek içeriği (YAML başlığı **hariç**) silin.

Görev A2 — YAML başlığını standarda uydurun

Belgenizin YAML başlığını aşağıdaki iskelete göre tamamlayın. Dört boşluk var.

```
---
title: "____"
author: "____"
date: today
lang: ____
format: ____
editor: visual
---
```

! Önemli

`lang` alanını boş bırakırsanız Quarto otomatik etiketleri İngilizce üretir: “Figure 1”, “Contents”, “Table 2”. Türkçe bir raporda bu, okuyucunun ilk fark ettiği şeydir.

Görev A3 — İlk render

Belgeyi render edin. PDF açılmalıdır.

Açılmadıysa **Background Jobs** / **Render** sekmesindeki hata mesajını okuyun. En sık iki neden: LaTeX kurulu değil (kurulum rehberine bakın) veya YAML’da girinti hatası var (`format:` ile `pdf` aynı hizada olmamalıdır).

Bölüm B — Kod öbekleri ve seçenekleri

Görev B1 — Paket yükleme öbeği

Belgenize bir kod öbeği ekleyin (**Ctrl+Alt+I**) ve içine `dplyr` ile `gapminder` paketlerini yükleyin. Bu öbeğin **ne kodu ne çıktısı** raporda görünmemelidir.

Hangi seçenek? Boşluğu doldurun.

```
#| label: paketler
#| ____: false

library(dplyr)
library(gapminder)
```

Görev B2 — Üç seçeneği ayırt edin

Aşağıdaki üç öbeği belgenize ekleyin ve render edin. Her birinin çıktıda **ne ürettiğini** not edin — üçü de farklıdır.

```
#| eval: false
mean(gapminder$lifeExp)
```

```
#| echo: false
mean(gapminder$lifeExp)
```

```
#| include: false
mean(gapminder$lifeExp)
```

Soru: Hangisi kodu gösterir ama sonucu göstermez? Hangisi hiçbir şey göstermez ama kodu yine de çalıştırır?

Görev B3 — Hata veren kod göstermek

Bir ders notunda “şu kod neden hata verir?” diye örnek göstermek istiyorsunuz. Aşağıdaki öbek render’ı **düşürmeden** hatayı belgede göstermelidir. Eksik seçeneği ekleyin.

```
#| ____: true
3 x 5
```

Bölüm C — Satır içi kod

Görev C1 — Sayıyı metne gömmek

Belgenize önce şu öbeği ekleyin:

```
#| include: false  
avrupa <- gapminder |> filter(year == 2007, continent == "Europe")
```

Sonra **metin** olarak şu cümleyi yazın ve boşlukları satır içi kodla doldurun:

2007 yılında veri setinde _____ Avrupa ülkesi vardır ve bu ülkelerin ortalama yaşam beklentisi _____ yıldır.

İpucu: satır içi kod, ters tırnak içinde **r** ile başlar.

Görev C2 — Neden elle yazmıyoruz?

avrupa nesnesini 2002 yılına göre yeniden tanımlayın ve belgeyi yeniden render edin. Cümledeki iki sayıya ne oldu?

```
#| eval: false  
avrupa <- gapminder |> filter(year == ____, continent == "Europe")
```

i Not

Bu, Quarto'nun en somut faydasıdır. Elle yazılmış bir sayı veri değişince **sessizce yanlış kalır**; satır içi kodla yazılmış bir sayı kalamaz.

Bölüm D — Tablolar

Görev D1 — Ham çıktıdan tabloya

Avrupa verisinden yaşam beklentisi en yüksek beş ülkeyi seçip önce ham çıktı olarak, sonra `knitr::kable()` ile yazdırın.

Görev E2 — Metinden atıf yapmak

Şekle metin içinden atıf yapan bir cümle yazın. Cümlede “yukarıdaki grafikte” geçmesin.

_____’te görüldüğü gibi, gelir arttıkça yaşam beklentisi artmakta ancak artış hızı yavaşlamaktadır.

Görev E3 — Numaranın kendiliğinden kayması

Bu şekilden **önce** başka bir etiketli şekil ekleyin (herhangi bir grafik olabilir). Render edin ve metindeki atıf numarasına bakın.

Elle numaralandırsaydınız ne olurdu?

Bölüm F — Sorun giderme

Görev F1 — Boş oturum tuzağı

Konsola şunu yazın (belgeye **değil**):

```
#| eval: false  
gizli_nesne <- 42
```

Sonra belgenize `gizli_nesne` yazan bir öbek ekleyip render edin.

Hata mesajı ne diyor? Neden konsolda çalışan kod belgede çalışmıyor?

Görev F2 — Yol tuzağı

Aşağıdaki iki satırdan hangisi arkadaşınızın bilgisayarında da çalışır? Neden?

```
#| eval: false  
veri <- read.csv("/Users/adiniz/Masaüstü/IST2083/data/titanic.csv")  
veri <- read.csv(here::here("data", "titanic.csv"))
```

Görev F3 — Belgeyi teslim etmek

Belgenizi son kez render edin ve klasörde oluşan dosyalara bakın: `ilk-raporum.qmd` ve `ilk-raporum.pdf`.

Soru: Bir arkadaşınızın raporunuzu yeniden üretebilmesi için ona hangisini göndermelisiniz? Ya karar vericiye?

Çözümler

Aşağıdaki kod çalıştırılır; çıktıları görmek için render edilmiş PDF'e bakabilir ya da satırları RStudio'da kendiniz çalıştırabilirsiniz.

A2 — YAML başlığı

```
---  
title: "İlk Raporum"  
author: "Adınız Soyadınız"  
date: today  
lang: tr  
format: pdf  
editor: visual  
---
```

`date: today` kullanıldı çünkü bu belgede hiç R öbeği olmayabilir; 2026-09-13 yalnızca knitr çalışıyorsa çözülür.

B1 — Paket öbeği

`include: false`. Öbek **çalışır** (paketler yüklenir) ama ne kodu ne çıktısı belgede görünür. `echo: false` olsaydı paket yükleme mesajları çıktıda görünürdü.

B2 — Üç seçeneğin farkı

Seçenek	Kod görünür mü?	Sonuç görünür mü?	Çalışır mı?
eval: false	evet	hayır	hayır
echo: false	hayır	evet	evet
include: false	hayır	hayır	evet

B3 — Hata göstermek

error: true. Bu olmadan 3 x 5 satırı render'ı tamamen düşürür.

C1 — Satır içi kod

```
library(dplyr)
library(gapminder)

avrupa <- gapminder |> filter(year == 2007, continent == "Europe")

nrow(avrupa)
```

[1] 30

```
round(mean(avrupa$lifeExp), 1)
```

[1] 77.6

Metinde şöyle yazılır (ters tırnaklar arasında r ile):

```
2007 yılında veri setinde 30 Avrupa ülkesi vardır ve bu
ülkelerin ortalama yaşam beklentisi 77.6
yıldır.
```

Render edilince: 2007 yılında veri setinde 30 Avrupa ülkesi vardır ve bu ülkelerin ortalama yaşam beklentisi 77.6 yıldır.

C2 — Yıl değişince

```
avrupa2002 <- gapminder |> filter(year == 2002, continent == "Europe")
c(ulke_sayisi = nrow(avrupa2002),
  ortalama    = round(mean(avrupa2002$lifeExp), 1))
```

```
ulke_sayisi    ortalama
          30.0         76.7
```

Ülke sayısı aynı kalır (veri setinde her ülke her yıl için vardır), ama ortalama değişir. Cümledeki sayı da kendiliğinden değişmiştir — siz hiçbir şey yapmadınız.

D1 ve D2 — Tablolar

```
en_uzun <- avrupa |>
  arrange(desc(lifeExp)) |>
  select(country, lifeExp, gdpPercap) |>
  head(5)

knitr::kable(
  en_uzun,
  digits    = 1,
  col.names = c("Ülke", "Yaşam beklentisi", "Kişi başı GSYİH"),
  caption   = "Yaşam beklentisi en yüksek beş Avrupa ülkesi, 2007"
)
```

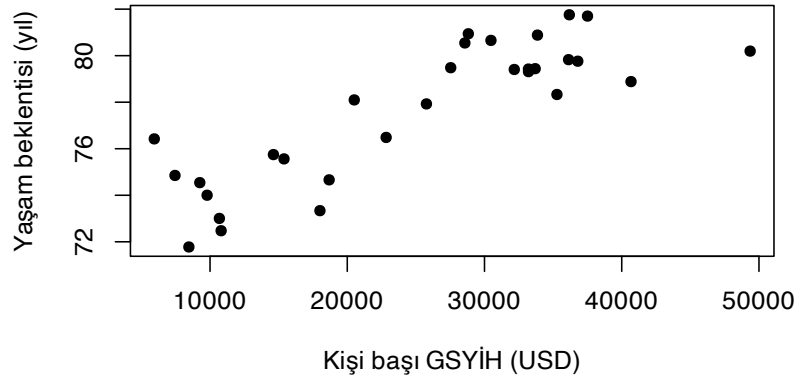
Tablo 2: Yaşam beklentisi en yüksek beş Avrupa ülkesi, 2007

Ülke	Yaşam beklentisi	Kişi başı GSYİH
Iceland	81.8	36180.8
Switzerland	81.7	37506.4
Spain	80.9	28821.1
Sweden	80.9	33859.7
France	80.7	30470.0

`digits = 1` olmasaydı `gdpPercap` sütunu altı basamaklı ondalıklarla basılırdı — tabloyu okunmaz yapan en yaygın kusur budur.

E1 ve E2 — Şekil ve atıf

```
plot(avrupa$gdpPercap, avrupa$lifeExp,  
      xlab = "Kişi başı GSYİH (USD)",  
      ylab = "Yaşam beklentisi (yıl)",  
      pch = 16)
```



Figür 1: Kişi başı gelir ile yaşam beklentisi ilişkisi, Avrupa 2007

Metinde @fig-gelir-yasam yazıldığında şu cümle elde edilir:

Figür 1’da görüldüğü gibi, gelir arttıkça yaşam beklentisi artmakta ancak artış hızı yavaşlamaktadır.

E3 — Numaranın kayması

Araya yeni bir etiketli şekil eklendiğinde tüm sonraki numaralar kendiliğinden bir artar ve metindeki atıflar da otomatik olarak güncellenir. Elle numaralandırırsaydınız, belgedeki her “Şekil 3” ifadesini tek tek bulup değiştirmeniz gerekirdi — ve birini kaçırdığınızda okuyucu yanlış şekle bakardı.

F1 — Boş oturum

Hata: object 'gizli_nesne' not found.

Quarto belgeyi **temiz, boş bir R oturumunda** render eder; konsolda tanımladığınız nesneleri görmez. Bu bir sınırlama değil, bir güvencedir: belge yalnızca kendi içinde tanımlanan şeylere dayanıyorsa başka bir bilgisayarda da aynı sonucu verir.

F2 — Yollar

İkincisi. `here::here()`, proje kökünü `IST2083.Rproj` dosyasından bulur; mutlak yol ise yalnızca o klasör yapısına sahip tek bir bilgisayarda çalışır.

F3 — Neyi göndermeli?

- **Arkadaşınıza / meslektaşınıza:** `.qmd` — yeniden üretebilmesi için kaynağa ihtiyacı vardır (ayrıca veriye de).
- **Karar vericiye:** `.pdf` — kodu değil sonucu ister.

Bu ayırım, Quarto’nun ders notunda geçen üç kullanım amacının pratikteki karşılığıdır.